

Вредное ПО:)





Что можно редактировать

Редактировать можно не только текстовые файлы, но и файлы в которых мы пишем наши программы (да и любые другие тоже). Но возникает вопрос? Если PyCharm может что-то редактировать может и любая другая программа может редактировать?

Да это действительно так, любая программа может редактировать файлы и вирусы это как раз пример таких программ. Они незаметно редактируют другие программы. Конечно ОС старается ограничить доступ к таким программам, но зловредный код умудряется обходить эти ограничения используя различные уязвимости в ОС или просто вводя пользователя в заблуждение

Так вредоносный код обходит различные ограничения нанося существенный материальный и моральный ущерб.





Пишем вирус и антивирус

Нашей задачей будет написать программу которая открывает соседнюю программу и дописывает в конце `print("Я вирус!")`

```
virus_code = 'print("Я вирус")'
```

```
with open('write_file.py', 'a', encoding='utf-8') as file:  
    file.write(f"\n{virus_code}\n")
```

>>>

```
with open('user_answers.txt', 'a', encoding='utf-8') as file:  
    for answer in user_answers:  
        file.write(f"{answer}\n")  
  
print("Ответы записаны!")  
  
print("Я вирус")
```

Как видим после выполнения нашей программы, в примере который был показан на странице записи данных в файл появилась новая строка которую мы сами там не добавляли

```
virus_code = 'print("Я вирус")'
```

```
with open('write_file.py', encoding='utf-8') as file:  
    content = file.read()
```

```
if virus_code in content:
```

```
    print("Вирус обнаружен!")
```

```
    new_content = content.replace('print("Я вирус")', '')
```

```
    with open('write_file.py', 'w', encoding='utf-8') as clean_file:
```

```
        clean_file.write(new_content)
```

```
else:
```

```
    print("Вирусов нет!")
```

А вот программа которая поможет нам от этого вируса избавиться, да возможно она действует не совсем изящно, по факту она копирует наш файл и если он инфицирован, вычищает строки с вирусным кодом, после чего перезаписывает файл. Но главное что это работает!

Перемещение и переименование файлов





Как можно переместить файл и присвоить ему новое имя?

Работу с файлами мы освоили, но часто возникает необходимость их переименовать или переместить. В этом помогут такие функции из модуля `os`:

- **`rename(path_1, path_2)`** – переименовывает и перемещает данные из `path_1` в `path_2`

```
import os
```

```
base_path = r"test_file.txt"
```

```
for path, dirnames, filenames in os.walk('new_dir'):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
```

```
new_path = r"new_dir\test_file_new_place.txt"
```

```
os.rename(base_path, new_path)
```

```
for path, dirnames, filenames in os.walk('new_dir'):
    print(f"path - {path}")
    print(f"dirnames - {dirnames}")
    print(f"filenames - {filenames}")
```

>>>

```
path - new_dir
```

```
dirnames - []
```

```
filenames - []
```

```
path - new_dir
```

```
dirnames - []
```

```
filenames - ['test_file_new_place.txt']
```

Как видим когда в программе запускается цикл в 1й раз то в папке **new_dir** у нас нет ничего.

После применения функции **`os.rename(base_path, new_path)`** в папке **new_dir** у нас появился файл **test_file_new_place.txt** который был перемещен из нашей корневой папки и переименован, при этом его содержимое не изменилось

module_10_files_in_OS

new_dir

files_06.py

files_07.py

files_08.py

files_09.py

test_file.txt

>>>

module_10_files_in_OS_renam

new_dir

test_file_new_place.txt

files_06.py

files_07.py

files_08.py

files_09.py

Здесь мы видим изменение файловой структуры нашего проекта, после применения функции **`rename()`** из модуля **`os`**



Как можно переместить файл и присвоить ему новое имя?

- `renames(path_1, path_2)` – делает тоже самое что и `rename()`, НО если директории становятся пустыми, удаляет их. Если необходимой директории нет, создает ее

```
import os
```

```
base_path = r"new_dir\test_file_new_place.txt"
```

```
for path, dirnames, filenames in os.walk('new_dir'):
```

```
    print(f"path - {path}")
```

```
    print(f"dirnames - {dirnames}")
```

```
    print(f"filenames - {filenames}")
```

```
new_path = r"new_dir_1\test_file_new_place_1.txt"
```

```
os.renames(base_path, new_path)
```

```
for path, dirnames, filenames in os.walk('new_dir_1'):
```

```
    print(f"path - {path}")
```

```
    print(f"dirnames - {dirnames}")
```

```
    print(f"filenames - {filenames}")
```

>>>

```
path - new_dir
```

```
dirnames - []
```

```
filenames - ['test_file_new_place.txt']
```

```
path - new_dir_1
```

```
dirnames - []
```

```
filenames - ['test_file_new_place_1.txt']
```

Как видим в папке **new_dir** у нас файл:

test_file_new_place.txt.

После применения функции **os.renames(base_path, new_path)**

у нас появилась папка **new_dir_1** в ней лежит файл **test_file_new_place.txt_1**, при этом папка **new_dir** отсутствует, т.к. она осталась пустой и поэтому была удалена.

module_10_files_in_OS_rename

new_dir

test_file_new_place.txt

files_06.py

files_07.py

files_08.py

files_09.py

>>>

module_10_files_in_OS_rename

new_dir_1

test_file_new_place_1.txt

files_06.py

files_07.py

files_08.py

files_09.py

Здесь мы видим изменение файловой структуры нашего проекта, после применения функции **renames()** из модуля **os**



Как можно перезаписать файл?

- **replace(path_1, path_2)** – переименовывает и перемещает данные из path_1 в path_2. Если данные с таким названием уже есть, перезаписывает их

```
import os
```

```
base_path = r"test_file_old_place_1.txt"
```

```
new_path = r"new_dir_1\test_file_new_place_1.txt"
```

```
os.replace(base_path, new_path)
```

test_file_old_place_1.txt x

1 some data in file:

2 test_file_OLD_place.txt

test_file_new_place_1.txt x

1 some data in file:

2 test_file_NEW_place.txt

>>>

test_file_new_place_1.txt x

1 some data in file:

2 test_file_OLD_place.txt

module_10_files_in_OS_rename

new_dir_1

test_file_new_place_1.txt

files_06.py

files_07.py

files_08.py

files_09.py

test_file_old_place_1.txt

>>>

module_10_files_in_OS_rename

new_dir_1

test_file_new_place_1.txt

files_06.py

files_07.py

files_08.py

files_09.py

У нас в разных папках есть 2 файла:

- **test_file_old_place.txt** в котором содержатся следующие данные:
some data in file:
test_file_OLD_place.txt
- **test_file_new_place.txt** в котором содержатся следующие данные:
some data in file:
test_file_NEW_place.txt

Мы вызываем функцию **replace(base_path, new_path)** и что мы видим в результате ее выполнения, у нас пропал файл **test_file_old_place.txt** и остался только файл **test_file_new_place.txt** !НО! содержимое файла **test_file_old_place.txt** теперь находится в файле **test_file_new_place.txt**, а это значит что файл **test_file_new_place.txt**, был **перезаписан!**

Как можно удалить файл и папку со всем содержимым?

- **remove(path)** – удаляет файл при его наличии, если файла не существует мы получим ошибку `FileNotFoundError` (обходим ошибку при помощи `try/except` если это необходимо).

```
import os

try:
    os.remove('test_path_1/file_to_remove.txt')
except FileNotFoundError:
    print("Файл не найден!")
else:
    print("Файл успешно удален!")
```

>>> Файл успешно удален!

▼ test_path_1

- ≡ file_to_remove.txt
- ≡ file_to_remove_1.txt
- ≡ file_to_remove_2.txt

>>>

▼ test_path_1

- ≡ file_to_remove_1.txt
- ≡ file_to_remove_2.txt

В папке `test_path` было 3 файла:

- `file_to_remove.txt`
- `file_to_remove_1.txt`
- `file_to_remove_2.txt`

Мы выполнили команду

`os.remove('test_path_1/file_to_remove.txt')`

В результате выполнения программы, файл который находился по указанному расположению был удален

Как можно удалить файл и папку со всем содержимым?

- для удаления директории со всем ее содержимым надо будет импортировать библиотеку **shutil** после чего ввести следующую команду **shutil.rmtree(path)**

```
import shutil
```

```
try:
```

```
    shutil.rmtree('test_path_1')
```

```
except FileNotFoundError:
```

```
    print("Директория не найдена!")
```

```
else:
```

```
    print("Удалена директория со всем содержимым!")
```

```
▼ test_path_1
```

```
    ≡ file_to_remove_1.txt
```

```
    ≡ file_to_remove_2.txt
```

```
🔗 06_01_01_rename_renames.py >>>
```

```
🔗 06_01_02_replace.py
```

```
🔗 06_01_03_remove.py
```

```
🔗 06_01_04_JSON.py
```

```
> json_data
```

```
🔗 06_01_01_rename_renames.py
```

```
🔗 06_01_02_replace.py
```

```
🔗 06_01_03_remove.py
```

```
🔗 06_01_04_JSON.py
```

>>> Удалена директория со всем содержимым!

В результате выполнения на данного кода, директория test_path_1 была успешно удалена, вместе со всем ее содержимым.



Как можно переместить файл и присвоить ему новое имя?

А для получения информации о дате создания или последней правки можно воспользоваться функциями:

- `getctime()` - время создания файла;
- `getmtime()` - время модификации, изменения файла;
- `getatime()` - время последнего обращения к файлу;
- `getsize()` - размер файла в байтах;

модуля `path`

```
import os

file_path = 'json_data/levels.json'

print(os.path.getctime(file_path))
print(os.path.getmtime(file_path))
print(os.path.getatime(file_path))
print(os.path.getsize(file_path))
```

>>> 1715788849.950652
1715788898.0559416
1715789323.5094714
99

Достаточно интересный результат а что он нам сообщает. Это время в секундах начиная с 00.00.00, 01.01.1970 г. Это считается датой начала эпохи Unix

Знакомство с JSON

JSON



Формат JSON

JSON это не родной формат для Python но близкий, это JSON это язык на котором frontend общается с backend. JSON это самый популярный текстовый формат во всем интернете!

И хотя бинарного кода передается больше почти каждый современный код использует JSON как транспорт для передачи данных с frontend на backend

Изначально формат был придуман программистами которые пишут на JavaScript поэтому у него такое и название, оно расшифровывается как:

- JavaScript
- Object
- Notation

Обозначение объектов JavaScript.

JSON



Формат JSON на что похож и зачем нужен?

Содержимое файла формата JSON выглядит следующим образом:

```
[
  {
    "employer": "Нетопия",
    "name": "Разработчик Python",
    "salary_from": null,
    "salary_to": 250000,
    "is_active": true
  },
  {
    "employer": "Радיום",
    "name": "Backend программист Python",
    "salary_from": 50000,
    "salary_to": null,
    "is_active": false
  }
]
```

И как видим по своей сути этот файл у нас похож на список словарей только это все текстовая строка, которую можно не только записать в файл а оттуда прочитать, но и переслать по интернету.

Чем дальше мы идем, тем с более сложными структурами данных мы работаем, мы уже знаем что можем хранить значения через пробелы, запятые, двоеточия, перенос строки. Но все эти разделители становятся проблемой когда в нашем контенте встречаются такие же пробелы, запятые или двоеточия.

Нужен такой формат данных который смог бы хранить:

- числа
- строки
- списки любой вложенности
- словари любой вложенности

Это все и есть JSON



Формат JSON на что похож и зачем нужен?

Содержимое файла формата JSON выглядит следующим образом:

```
[
  {
    "employer": "Нетопия",
    "name": "Разработчик Python",
    "salary_from": null,
    "salary_to": 250000,
    "is_active": true
  },
  {
    "employer": "Радיום",
    "name": "Backend программист Python",
    "salary_from": 50000,
    "salary_to": null,
    "is_active": false
  }
]
```

Так какие отличия от словарей в Python у формата JSON:

- **ключом может быть только строка;**
- **всегда используются только двойные кавычки;**
- **после последнего элемента не ставится запятая;**
- **нельзя использовать кортежи и множества - только;**
списки и словари;
- **true и false - пишется с маленькой буквы;**
- **пустое значение не None а null.**



Превращение в JSON

В python мы можем наши словари записать в JSON формат при помощи следующих действий:

1. `import json`
2. `json.dumps()` - где в скобки мы передаем наш словарь

Например:

```
import json

levels = {
    0: "Нулевой",
    1: "Так себе",
    2: "Можно лучше",
    3: True,
    4: False,
    5: None,
}

levels_json = json.dumps(levels, ensure_ascii=False)

with open('levels.json', 'w', encoding='utf-8') as file:
    file.write(levels_json)

print(levels_json)
```

>>>

levels.json ×

```
1 {
2     "0": "Нулевой",
3     "1": "Так себе",
4     "2": "Можно лучше",
5     "3": true,
6     "4": false,
7     "5": null
8 }
```

>>>

```
{"0": "Нулевой", "1": "Так себе", "2": "Можно лучше", "3": true, "4": false, "5": null}
```



А теперь наоборот из JSON в Python

В python объекты JSON могут быть загружены напрямую как словари Python с помощью метода **json.loads()** из модуля json. После загрузки JSON в качестве словаря операции над объектом идентичны операциям с обычным объектом словаря.

```
import json

base_json = """
{
    "0": "Нулевой",
    "1": "Так себе",
    "2": "Можно лучше",
    "3": true,
    "4": false,
    "5": null
}
"""

python_dict = json.loads(base_json)
print(python_dict)
```

>>> {'0': 'Нулевой', '1': 'Так себе', '2': 'Можно лучше', '3': True, '4': False, '5': None}